

Confluent Kafka Isotope

An example of using Kafka **consumer and producer interceptors** to tag every message with a tracer — an *isotope* — that carries through every hop of a multi-topic pipeline. Apache Flink consumes the tagged records and reports on what the isotopes reveal: **end-to-end latency**, **hop topology**, **drop/duplication rates**, and **pipeline coverage**.

A **portability requirement** runs through this project: the same isotope mechanism must work against both **Confluent Cloud for Apache Flink (CCAF)** (managed) and **Confluent Platform for Apache Flink** (self-managed) — both used here via Table API SQL plus uploaded UDF/PTF JARs (no DataStream code on either side). Three decisions follow from that: tagging happens in the Kafka **producer/consumer interceptors** (the one extension point both runtimes share via the broker), the on-wire **header** format is **JSON** (so Flink SQL can read the scalar fields with `CAST(headers [...] AS STRING)` and no UDF), and the optional stateful reports (`LatencyPercentilesUDAF`, `StuckTracePTF`) ship as a single JAR that registers identically on either runtime.

One asymmetry the runtimes don't share: **Flink-native SR-Protobuf**. CCAF supports SR-framed Protobuf as a Flink sink format via its topic catalog; Apache Flink open-source (the CP Flink runtime) ships `avro-confluent` but no SR-Protobuf counterpart. So Flink *report sinks* land on **Avro+SR on CP** and can be **Protobuf+SR on CCAF** — a runtime constraint, not a project preference. The demo *event* topics (next paragraph) are unaffected because they're written by the Kafka producer client, not by Flink.

Message **values** on the demo topics are **SR-framed Protobuf** (`ai.signalroom.kafka.isotope.proto.DemoEvent`) — the standard Confluent value format. The interceptors and reports are agnostic to value format because the isotope rides in headers; the Protobuf choice just gives the integration tests and the demo CLI a typed payload to work with.

Table of Contents

- [1.0 How the isotope is carried](#)
 - [2.0 Repo layout](#)
 - [3.0 Running](#)
 - [3.1. Unit tests \(no broker, instant\)](#)
 - [3.2 Demo CLI — see one trace propagate live](#)
 - [3.3 Integration tests \(live Kafka via Minikube\)](#)
 - [3.4 Flink SQL reports on Minikube](#)
 - [3.5 Recommended path the first time through](#)
-

1.0 How the isotope is carried

- **Header `x-isotope`** (JSON bytes) carries the full hop history, forwarded by every hop:
 - `t` — 16-byte **UUIDv7** trace ID (RFC 9562): 48-bit ms timestamp in the high bits + 74 bits random. Stable for the life of the trace, and lexicographic byte order matches creation order — sort trace IDs and you get chronological order for free.

- **o** — origin timestamp (ms) — same value as the timestamp embedded in the UUIDv7 trace ID; kept as its own field for typed access from Flink SQL without needing to decode the UUID bytes.
- **s** — origin service name (set once, never reassigned)
- **h** — ordered list of hops, each `{s: service, t: topic, m: tsMs}`
- **x** — **true** if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted
- **Six scalar headers** (UTF-8 strings) carry the most-recent-hop view so Flink SQL can read them via `CAST(headers['x-isotope-...'] AS STRING)` without parsing the JSON array (no UDF required on either CCAF or CP Flink). See [flink/README.md](#) for the full header table.

A producer with the isotope interceptor loaded appends one hop on every `send()`. A consume-then-produce service calls `IsotopeContext.adoptFromRecord(record)` between consume and produce so the trace ID and origin survive the hop.

2.0 Repo layout

app/	isotope JVM library + demo CLI +
tests	
src/main/proto/	DemoEvent.proto (Protobuf message
value)	
src/main/java/ai/signalroom/kafka/isotope/	
Isotope.java	POJO + JSON codec + Hop +
fromHeaders()	
	+ UUIDv7 helpers (uuidV7Bytes /
uuidV7String)	
IsotopeContext.java	ThreadLocal + adoptFromRecord()
IsotopeProducerInterceptor.java	stamps/appends x-isotope + 6 scalar
	reporting headers on send()
IsotopeConsumerInterceptor.java	batch-aware logging; no auto-
propagation	
App.java	demo CLI — send / hop / sink modes
src/test/java/.../	IsotopeCodecTest (no broker needed)
src/integrationTest/java/.../	live-broker tests; produce/consume
	DemoEvent via SR-framed Protobuf
	(need Minikube CP + SR port-
forwarded)	
ptf/	Phase 2 — Flink PTF + UDAF shadow
JAR	
src/main/java/ai/signalroom/kafka/isotope/flink/	
LatencyPercentilesUDAF.java	T-Digest p50/p95/p99 aggregate
StuckTracePTF.java	per-trace state + event-time timer
Percentiles.java, StuckTraceAlert.java	return-type DT0s
src/test/java/.../	LatencyPercentilesUDAFTest
flink/sql/	Flink SQL reports — identical on
	CCAF and CP Flink except source DDL
cp/	CP Flink: 00_source_table,
01_register_functions,	
	05_report_sinks (avro-confluent),
99_teardown	
cc/	CCAF: 00_source_table (view over
auto-registered	

```
(confluent-artifact:// JAR)
  shared/
reports
30_hop_distribution,
70_latency_percentiles)
k8s/base/
manifests
scripts/
reports.sh
Makefile
...

topic), 01_register_functions
05_isotope_view + 6 INSERT INTO
(10_latency, 20_topology,
40_coverage, 60_stuck_trace,
CFK Kafka/SR/Connect/ksqlDB/C3
port-forward helpers, deploy-flink-
cp-up / flink-up / kafka-pf-up /
```

3.0 Running

3.1. Unit tests (no broker, instant)

```
./gradlew test # both subprojects
# or scoped:
./gradlew :app:test # IsotopeCodecTest - JSON
roundtrip, hop eviction, header size, UUIDv7 properties (10 tests)
./gradlew :ptf:test # LatencyPercentilesUDAFTest - T-
Digest accumulator semantics
```

3.2 Demo CLI — see one trace propagate live

The fastest way to watch the isotope mechanic. Requires the cluster to be up and the Kafka + SR forwards running (see step 3 below for the bring-up commands). The CLI has three modes:

Mode	Args	What it does
send	<topic> <service> <payload>	Produces one isotope-tagged DemoEvent to <topic>, then exits. Auto-creates the topic.
hop	<in-topic> <out- topic> <service>	Consumes records from <in-topic>, adopts the isotope into thread-local, re-produces the same DemoEvent to <out-topic> as <service> (which appends a new hop). Runs until Ctrl-C.
sink	<topic>	Subscribes to <topic> and pretty-prints the full isotope trail for every arriving record. Runs until Ctrl-C.

A 3-stage chain in four terminals:

```
# Terminal A - terminal sink (will print the full 3-hop trail)
./gradlew :app:run --args="sink iso-final" -q
```

```
# Terminal B – middle stage: iso-mid → iso-final as svc-C
./gradlew :app:run --args="hop iso-mid iso-final svc-C" -q

# Terminal C – first stage: iso-start → iso-mid as svc-B
./gradlew :app:run --args="hop iso-start iso-mid svc-B" -q

# Terminal D – kick the chain off (run repeatedly to send more)
./gradlew :app:run --args="send iso-start svc-A 'hello world'" -q
```

Terminal A's output for each record shows the same `trace_id` across all three hops, `origin = svc-A` (never reassigned), and `hops[]` listing `svc-A → svc-B → svc-C` in order with per-hop timestamps. Override endpoints via `-Dkafka.bootstrap=...` / `-Dschema.registry.url=...` if you're not on the default Minikube layout.

3.3 Integration tests (live Kafka via Minikube)

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start          # one-time
make cp-up                   # CFK Operator +
Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up             # localhost:30092 → Kafka,
localhost:8081 → Schema Registry
```

Then run the suite:

```
./gradlew :app:integrationTest          #
all 5 tests
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT'  #
just one
```

Override the endpoints if needed:

```
./gradlew :app:integrationTest \
  -PkafkaBootstrap=localhost:30092 \
  -PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
<code>BrokerSmokeIT</code>	AdminClient can create/list/delete a topic via the NodePort port-forward
<code>ProducerInterceptorIT</code>	A bare consumer sees the <code>x-isotope</code> JSON header + all 6 scalar reporting headers with the expected origin/hop values, and the Protobuf round-trip preserves <code>DemoEvent.source / payload</code>
<code>ConsumerInterceptorIT</code>	<code>IsotopeContext.adoptFromRecord</code> extracts isotope into thread-local on tagged records; clears the thread-local for untagged records
<code>ThreeStageHopPropagationIT</code>	<code>svc-A → topic-AB → svc-B → topic-BC → svc-C</code> produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = <code>svc-A</code> , this = <code>svc-B</code> , hop count = 2) at the terminal

3.4 Flink SQL reports on Minikube

The four Phase-1 reports plus the Phase-2 PTF and UDAF reports — six in total — run against a Flink session cluster managed by the Confluent Flink Kubernetes Operator. Same FQL files deploy to Confluent Cloud for Apache Flink — see [flink/README.md](#) for that side; this section is the local-Minikube path.

Bring up Flink:

```
make flink-up          # cert-manager → CFK Flink Operator → CMF →
session cluster        # (~5 min the first time)
```

Deploy the reports — all 6 reports run on the cp-flink session cluster (Flink 2.1.2). Sink topics use Apache Flink's `avro-confluent` format — SR-framed Avro, auto-registered on first write — so Control Center renders the report rows natively.

```
make flink-reports-up
```

`flink-reports-up` builds the PTF/UDAF shadow JAR if missing, copies it into the JobManager pod, pre-creates the 6 sink Kafka topics, then applies the source + view + sink DDL and submits 6 `INSERT INTO` streaming jobs (one per report). On first write to each sink, Apache Flink's `flink-sql-avro-confluent-registry` format registers a fresh Avro schema in SR under subject `<topic>-value`. **Control Center deserializes all 6 report topics natively** — no `.proto` files in the repo for the reports, no hand-installed format jars beyond the one init-container download.

3.4.1 Format-by-domain

The demo event topics (*iso-start*, *iso-mid*, *iso-final*) still ride **Protobuf+SR** via the Java app's **DemoEvent** schema — that's unchanged. The *report* topics ride **Avro+SR** because cp-flink doesn't ship an SR-integrated Protobuf format and CMF (which does) disallows the UDAFs the percentiles report needs. Events from the app are Protobuf; aggregates from Flink are Avro. Two formats by domain — a clean split, not a defect.

3.5 Recommended path the first time through

1. `./gradlew test` — proves the codec + UDAF logic without any cluster.
2. `make cp-up && make kafka-pf-up && ./gradlew :app:integrationTest` — proves the broker + SR + interceptor + Protobuf path end-to-end.
3. The 3-stage demo CLI walkthrough above — visually shows the trace accumulating hops.
4. `make flink-up && make flink-reports-up && make flink-sql` — reports populate as you drive traffic via the demo CLI (see § 3.2).